



**UNIVERSIDAD TECNOLÓGICA
DE QUERÉTARO**

Alumna:

Vasquez Noyola Andrea

Grupo:

IDGS17

Matrícula:

2023148002

Materia:

Seguridad en el Desarrollo de Aplicaciones

Práctica SonarQube

1. ¿Qué diferencia hay entre un Security Hotspot y un Bug en SonarQube?

Un bug es un error lógico en el código que causa que la aplicación falle, se comporte de forma inesperada o rompa la funcionalidad. Es un problema que debe corregirse inmediatamente porque afecta el funcionamiento; mientras que un Security Hotspot: Es un fragmento de código sensible a la seguridad que podría ser una vulnerabilidad, pero requiere que un desarrollador lo revise manualmente para confirmar si representa un riesgo real en ese contexto específico. No siempre es un error, a veces es solo una advertencia para validar que se implementó de forma segura.

2. ¿Por qué es inseguro concatenar directamente el input del usuario en una query SQL? Explica el ataque con un ejemplo de payload.

Es inseguro porque la base de datos no puede distinguir entre cuál es la instrucción original que escribió el programador y cuál es la información que ingresó el usuario. Al concatenar, el atacante puede inyectar comandos de SQL que alteren la lógica de la consulta.

Por ejemplo, tenemos el caso de las prácticas anteriores del ' OR '1'='1 en donde si la consulta original es `SELECT * FROM Users WHERE Username = ' + input + '`, al ingresar el payload, la consulta se transforma en `SELECT * FROM Users WHERE Username = " OR '1'='1'`. Como `1=1` siempre es verdadero, la base de datos ignora la validación de usuario y devuelve todos los registros, permitiendo un bypass de autenticación.

3. ¿Qué es el Code Coverage y por qué SonarQube lo considera relevante para la seguridad?

El Code Coverage o cobertura de código es el porcentaje del código fuente que se ejecuta y se valida durante las pruebas automatizadas. Su relevancia para la seguridad es que SonarQube lo considera vital porque el código que no está cubierto por pruebas es un "punto ciego". Si las validaciones de seguridad (como revisar si un usuario tiene permisos) no tienen pruebas, un desarrollador podría borrarlas o romperlas accidentalmente en el futuro sin que el sistema avise, introduciendo vulnerabilidades sin darse cuenta.

4. Si un colega sube el token de SonarQube a GitHub por error, ¿qué pasos seguirías para mitigar el riesgo?

1. **Revocar el token inmediatamente:** Entrar a la configuración de SonarQube y eliminar ese token comprometido para invalidar cualquier intento de acceso.
2. **Generar un nuevo token:** Crear uno nuevo exclusivamente para el proceso que lo necesite.
3. **Limpiar el historial de Git (Opcional pero recomendado):** Usar herramientas como `git filter-repo` o `BFG` para borrar el token de todos los commits anteriores, ya que borrarlo solo en el último commit igual lo deja visible en el historial.

4. **Implementar buenas prácticas:** Asegurarse de que el nuevo token se guarde como Variable de Entorno o en un Gestor de Secretos (como GitHub Secrets), y nunca directamente en los archivos de configuración o en el código fuente .

5. ¿Cómo integrarías SonarQube en un pipeline CI/CD de GitHub Actions? Describe los pasos a alto nivel.

1. **Configurar Secretos:** Guardar la URL del servidor (SONAR_HOST_URL) y el Token (SONAR_TOKEN) en la sección de "Secrets" del repositorio en GitHub para no exponerlos.
2. **Crear el archivo Workflow:** Añadir un archivo .yml en la carpeta .github/workflows del proyecto.
3. **Preparar el entorno:** Definir pasos en el pipeline para instalar el SDK de .NET y Java (necesario para el escáner de Sonar).
4. **Instalar SonarScanner:** Agregar un paso para instalar la herramienta global (dotnet tool install --global dotnet-sonarscanner).
5. **Ejecutar el análisis:** Ejecutar los comandos begin, build, test y end, pasándole las credenciales mediante la sintaxis de secretos de GitHub (ej. \${secrets.SONAR_TOKEN}).